

GUÍA DE ESTUDIO · PROGRAMACIÓN WEB

Manual de JavaScript Avanzado

Arrays · Objetos · Métodos funcionales · Patrones · Observers

 filter() · map() · reduce()

 every() · some() · find()

 Matrices y Arrays anidados

 Objetos y métodos

 Funciones de orden superior

 Patrón Observer

 Web APIs: MutationObserver

 IntersectionObserver

 PerformanceObserver

Categoría de Programación Web · Septiembre 2026



Índice de Contenidos

MÓDULO 1 — ARRAYS Y MÉTODOS FUNCIONALES

1.1	filter() y map() encadenados	3
1.2	every() — validación de todos los elementos	4
1.3	reduce() — acumulaciones y agrupaciones	5
1.4	sort() — ordenamiento numérico y estable	6
1.5	flat() — aplanado de arrays anidados	6

MÓDULO 2 — MATRICES Y ALGORITMOS

2.1	Suma de filas y columnas de una matriz	7
2.2	Diagonales principal y secundaria	8
2.3	Número más repetido con frecuencias	9

MÓDULO 3 — OBJETOS EN JAVASCRIPT

3.1	Objetos literales con métodos y this	10
3.2	Object.entries(), values(), fromEntries()	11
3.3	Comparación profunda — deepEqual()	12
3.4	Fusión recursiva — merge()	12
3.5	Inspección y búsqueda recursiva de objetos	13

MÓDULO 4 — FUNCIONES DE ORDEN SUPERIOR

4.1	Funciones como parámetros y retorno	14
4.2	Cifrado Caesar y transformación de texto	14

MÓDULO 5 — PATRÓN OBSERVER Y WEB APIS

5.1	Patrón Observer — clase Subject	15
5.2	MutationObserver — To-Do List reactiva	16
5.3	IntersectionObserver — animaciones on-scroll	17
5.4	PerformanceObserver — monitor de rendimiento	18

Arrays y Métodos Funcionales

filter, map, every, reduce, sort, flat — programación declarativa sobre colecciones

1.1 — filter() y map()

FUNDAMENTO

■ CONCEPTO

filter(fn) recorre el array y devuelve un *nuevo array* con los elementos que cumplan la condición. **map(fn)** transforma cada elemento según la función dada, devolviendo otro nuevo array. Ambos son *inmutables*: no modifican el array original.

💡 Tip — Encadenamiento

Como cada uno devuelve un nuevo array, se pueden **encadenar directamente**:

```
arr.filter(...).map(...) . Esto es el núcleo de la programación funcional en JavaScript.
```

| Sintaxis

```
const resultado = array
  .filter((item) => condicion) // mantiene los que devuelven true
  .map((item) => transformacion); // transforma cada uno
```

| Ejercicio resuelto — Productos con stock



Filtrar productos disponibles y resumirlos

Dado un array de productos con nombre, precio y stock, obtener solo los que tienen stock mayor a 0 y proyectar únicamente nombre y precio.

```
const productos = [
  { nombre: "Notebook",    precio: 1200, stock: 5 },
  { nombre: "Mouse",       precio: 25,   stock: 0 },
  { nombre: "Teclado",     precio: 45,   stock: 10 },
  { nombre: "Monitor",     precio: 350,  stock: 3 },
];

// Paso 1: filtrar los que tienen stock
const conStock = productos.filter((p) => p.stock > 0);

// Paso 2: proyectar solo nombre y precio
const resumen = conStock.map((p) => ({ nombre: p.nombre, precio: p.precio }));

// Versión encadenada (equivalente)
const resumenEncadenado = productos
  .filter((p) => p.stock > 0)
  .map((p) => ({ nombre: p.nombre, precio: p.precio }));
```

Tabla de referencia

Método	Devuelve	Muta original	Caso de uso típico
<code>filter(fn)</code>	Array filtrado	No	Subsección de datos según criterio
<code>map(fn)</code>	Array transformado	No	Proyectar propiedades, transformar valores
<code>filter().map()</code>	Array filtrado+transformado	No	Pipeline de datos más común en JS

1.2 — every() VALIDACIÓN

■ CONCEPTO

every(fn) devuelve `true` solamente si *todos* los elementos cumplen la condición. En cuanto encuentra uno que no la cumple, **corta la ejecución** y retorna `false`. Su opuesto es **some(fn)**, que retorna `true` si al menos uno cumple.



Validar array de empleados con every()

Verificar que todos los empleados tienen nombre, salario válido, pertenecen a un departamento permitido y si son de IT tienen al menos 2 años de experiencia.

```
const departamentosValidos = ["IT", "Ventas", "Marketing"];

// Todos con nombre no vacío
const todosConNombre = empleados.every(
  (emp) => emp.nombre && emp.nombre.trim() !== ""
);

// IT deben tener ≥ 2 años de experiencia
// Si NO es IT → la condición se ignora (true)
const itConExp = empleados.every(
  (emp) => emp.departamento !== "IT" || emp.experiencia ≥ 2
);

// Verificación global: todas las condiciones en un solo every()
const todosValidos = empleados.every(
  (emp) =>
    emp.nombre && emp.nombre.trim() !== "" &&
    emp.salario > 500000 &&
    departamentosValidos.includes(emp.departamento) &&
    (emp.departamento !== "IT" || emp.experiencia ≥ 2)
);
```

💡 Patrón clave

Para condiciones condicionales del tipo "si X entonces Y", usá el patrón `!X || Y`. Si el elemento no aplica a la condición, el primer operando es `true` y se cortocircuita.

1.3 — reduce() ACUMULACIÓN

■ CONCEPTO

reduce(fn, valorInicial) recorre el array acumulando un único resultado. La función recibe (`acumulador`, `elementoActual`) y lo que devuelva se convierte en el nuevo acumulador. Es el más potente: puede reemplazar a filter, map y sum.

```
// Sintaxis general
const resultado = array.reduce((acc, item) => {
  // acc = acumulador   item = elemento actual
  return nuevoAcc;
}, valorInicial);
```



Suma de valores

```
arr.reduce((acc, n) => acc + n, 0)
```



Agrupar por clave

Acumulador = objeto `{}` ; agregar al array de cada clave con `push()`



Conteo de frecuencias

```
acc[n] = (acc[n] || 0) + 1
```



Máximo / Mínimo

```
(mejor, actual) => actual > mejor ? actual : mejor
```



EJERCICIO 1.3 · A4_3.JS + A4_4.JS

Agrupación y frecuencias con `reduce()`

Agrupar empleados por departamento y encontrar el número más repetido de un array.

```
// Agrupar empleados por departamento
const agrupados = empleados.reduce((acc, emp) => {
  const dpto = emp.departamento;
  if (!acc[dpto]) acc[dpto] = []; // inicializar si no existe
  acc[dpto].push(emp.nombre);
  return acc;
}, {});

// Número más repetido
const frecuencias = numeros.reduce((acc, num) => {
  acc[num] = (acc[num] || 0) + 1;
  return acc;
}, {});

const [masRepetido, veces] = Object.entries(frecuencias).reduce(
  (mejor, actual) => actual[1] > mejor[1] ? actual : mejor
);
```

1.4 — `sort()` y `flat()`

ORDEN Y APLANADO

⚠ Atención — `sort()` muta el array original

A diferencia de `filter/map/reduce`, **`sort()` modifica el array en el lugar**. Para no mutar el original, copialo primero con spread: `[...array].sort(...)`.



Ordenar estudiantes y aplanar matrices 3D

```
// sort() numérico descendente (sin mutar el original)
const ordenados = [...estudiantes].sort((a, b) => b.nota - a.nota);

// flat() - aplanar una matriz 3D
const matriz3D = [ [[3,1],[4,1]], [[5,9],[2,6]], [[5,3],[8,9]] ];
const aplanado = matriz3D.flat(2); // 2 niveles de anidamiento
const resultado = matriz3D.flat(Infinity).sort((a, b) => a - b); // encadenado
```

Método sort()	Comparador	Resultado
sin comparador	—	Ordena como strings ⚠️ bug con números
<code>(a, b) => a - b</code>	numérico	Ascendente (menor a mayor)
<code>(a, b) => b - a</code>	numérico	Descendente (mayor a menor)
<code>(a, b) => a.prop - b.prop</code>	por propiedad	Objetos por campo numérico

Matrices y Algoritmos

Trabajar con arrays bidimensionales usando `map()` y `reduce()` anidados

2.1 — Suma de filas y columnas

■ CONCEPTO

Una **matriz** en JS es un array de arrays. Para sumar filas se aplica `reduce()` dentro de `map()`. Para columnas, se itera por índice usando el segundo parámetro de `map()`.

12
34

EJERCICIO 2.1 · A4_42.JS

Suma de filas y columnas de una matriz 3×3

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

// Suma de cada FILA → map sobre filas, reduce suma elementos
const sumaFilas = matriz.map(
  (fila) => fila.reduce((acc, n) => acc + n, 0)
); // → [6, 15, 24]

// Suma de cada COLUMNA → iterar por índice j
const sumaColumnas = matriz[0].map((_, j) =>
  matriz.reduce((acc, fila) => acc + fila[j], 0)
); // → [12, 15, 18]
```

💡 Truco del índice `_`

Cuando en `map((valor, indice))` no necesitás el valor sino el índice, usá `_` como nombre convencional del parámetro ignorado.

2.2 — Diagonales de una matriz nxn



Diagonal principal y secundaria de cualquier matriz cuadrada

La **diagonal principal** son los elementos donde fila === columna $\rightarrow$ `[i][i]` . La **diagonal secundaria** está donde fila + columna = n-1 $\rightarrow$ `[i][n-1-i]` .

```
function analizarDiagonales(matriz) {  
  const n = matriz.length;  
  
  const diagPrincipal = matriz.map((fila, i)  $\Rightarrow$  fila[i]);  
  const diagSecundaria = matriz.map((fila, i)  $\Rightarrow$  fila[n - 1 - i]);  
  
  const sumaPrincipal = diagPrincipal.reduce((a, v)  $\Rightarrow$  a + v, 0);  
  const sumaSecundaria = diagSecundaria.reduce((a, v)  $\Rightarrow$  a + v, 0);  
  
  // Si n es impar el centro pertenece a AMBAS diagonales  $\rightarrow$  restarlo una vez  
  const centro = n % 2  $\neq$  0 ? matriz[Math.floor(n/2)][Math.floor(n/2)] : 0;  
  const sumaAmbas = sumaPrincipal + sumaSecundaria - centro;  
  
  return { diagPrincipal, diagSecundaria, sumaPrincipal, sumaSecundaria, sumaAmbas };  
}
```

Objetos en JavaScript

Objetos literales, métodos, this, Object.entries/values/fromEntries, recursión

3.1 — Objetos literales y el operador

[this](#)[BASE](#)

■ CONCEPTO

Un **objeto literal** en JS es una colección de pares clave-valor entre llaves `{ }`. Los valores pueden ser primitivos, arrays, otros objetos o **funciones (métodos)**. Dentro de un método, `this` hace referencia al propio objeto.



EJERCICIO 3.1 · A4_441.JS

Objetos con métodos — Libro, Cuenta Bancaria, Rectángulo

```
// Objeto con métodos que usan this
const cuentaBancaria = {
  titular: "María Gómez",
  saldo: 100000,

  depositar(monto) {
    if (monto ≤ 0) { console.log("Monto inválido"); return; }
    this.saldo += monto;
    console.log(`Depósito: +${monto}. Saldo: ${this.saldo}`);
  },

  retirar(monto) {
    if (monto > this.saldo) { console.log("Fondos insuficientes"); return; }
    this.saldo -= monto;
    console.log(`Retiro: -${monto}. Saldo: ${this.saldo}`);
  },
};
```

3.2 — Object.entries(), values(), fromEntries()

[ITERACIÓN](#)

■ CONCEPTO

Estas tres funciones de `Object` permiten tratar los objetos como arrays para poder aplicarles `filter/map/reduce`.

Método	Devuelve	Ejemplo de uso
<code>Object.keys(obj)</code>	Array de claves	Iterar propiedades
<code>Object.values(obj)</code>	Array de valores	Sumar, reducir valores
<code>Object.entries(obj)</code>	Array de [clave, valor]	filter/map con destructuring
<code>Object.fromEntries(arr)</code>	Objeto desde pares	Reconstruir después de map()



EJERCICIO 3.2 · A4_441.JS — INVENTARIO

Manipular un inventario con `Object.entries/values/fromEntries`

```
const inventario = {
  teclado: { precio: 50000, stock: 10 },
  mouse:   { precio: 25000, stock: 20 },
  monitor: { precio: 180000, stock: 5 },
};

// Valor total del inventario
const valorTotal = Object.values(inventario)
  .reduce((acc, p) => acc + p.precio * p.stock, 0);

// Incrementar precio 15% sin mutar el original
const actualizado = Object.fromEntries(
  Object.entries(inventario).map(([nombre, prod]) => [
    nombre,
    { ...prod, precio: Math.round(prod.precio * 1.15) },
  ])
);
```

3.3 — `deepEqual()` y `merge()` recursivos AVANZADO



Comparar y fusionar objetos de forma recursiva

```
// deepEqual: compara dos objetos en profundidad
function deepEqual(obj1, obj2) {
  if (obj1 === obj2) return true;
  if (typeof obj1 !== "object" || obj1 === null ||
      typeof obj2 !== "object" || obj2 === null) return false;

  const claves1 = Object.keys(obj1);
  const claves2 = Object.keys(obj2);
  if (claves1.length !== claves2.length) return false;

  return claves1.every(
    (clave) => claves2.includes(clave) && deepEqual(obj1[clave], obj2[clave])
  );
}

// merge: fusión recursiva (obj2 tiene prioridad)
function merge(obj1, obj2) {
  const res = { ...obj1 }; // copia superficial de obj1
  for (const clave in obj2) {
    const ambosObj =
      typeof obj1[clave] === "object" && obj1[clave] !== null &&
      typeof obj2[clave] === "object" && obj2[clave] !== null;
    res[clave] = ambosObj ? merge(obj1[clave], obj2[clave]) : obj2[clave];
  }
  return res;
}
```

Funciones de Orden Superior

Funciones que reciben o devuelven otras funciones — composición y reutilización

4.1 — ¿Qué es una función de orden superior?

CONCEPTO CLAVE

■ CONCEPTO

En JavaScript las funciones son **ciudadanos de primera clase**: se pueden asignar a variables, pasar como argumentos y retornar desde otras funciones. Una **función de orden superior** es aquella que *recibe* una función como parámetro o *devuelve* una función.



Recibe funciones

Ej: `map(fn)`, `filter(fn)`, `sort(fn)`, funciones propias que aceptan callbacks



Devuelve funciones

Ej: fábricas de funciones, currying, memoización



Capitalizar nombres y encriptar claves — Cifrado Caesar

Definir funciones independientes y pasarlas como argumentos a una función de transformación de usuarios.

```
// Capitalizar cada palabra de un string
function capitalizar(str) {
  return str.split(" ")
    .map((w) => w.charAt(0).toUpperCase() + w.slice(1).toLowerCase())
    .join(" ");
}

// Cifrado Caesar: desplazar cada letra +3 en el abecedario
function encriptar(str) {
  return str.split("").map((char) => {
    if (char ≥ "a" && char ≤ "z")
      return String.fromCharCode((char.charCodeAt(0) - 97 + 3) % 26 + 97);
    return char;
  }).join("");
}

// Función de orden superior que recibe dos funciones como parámetros
function transformarUsuarios(arr, fnNombre, fnClave) {
  return arr.map((u) => ({
    nombre: fnNombre(u.nombre),
    clave: fnClave(u.clave),
  }));
}

// Llamada: las funciones se pasan sin paréntesis (son referencias)
const resultado = transformarUsuarios(usuarios, capitalizar, encriptar);
```

Patrón Observer y Web APIs

Subject/Observer, MutationObserver, IntersectionObserver, PerformanceObserver

5.1 — Patrón Observer DISEÑO

■ CONCEPTO

El **patrón Observer** define una relación *1 a muchos*: cuando el **Subject** cambia de estado, notifica automáticamente a todos sus **Observers** suscritos. Es la base del manejo de eventos en toda interfaz reactiva.

Componente	Rol	Métodos clave
Subject	Emisor — gestiona la lista	subscribe(), unsubscribe(), notify()
Observer	Suscriptor — reacciona	Cualquier función(data) ⇒ void



Implementar la clase Subject

```
class Subject {
  constructor() {
    this._observers = []; // _ → convención "privado"
  }

  subscribe(observer) {
    if (this._observers.includes(observer)) { return; } // evitar duplicados
    this._observers.push(observer);
  }

  unsubscribe(observer) {
    this._observers = this._observers.filter((obs) => obs !== observer);
  }

  notify(data) {
    this._observers.forEach((observer) => observer(data));
  }

  get observerCount() { // getter: se accede sin paréntesis
    return `Cantidad: ${this._observers.length}`;
  }
}
```

5.2 — MutationObserver

WEB API

■ CONCEPTO

MutationObserver es una API nativa del navegador que detecta cambios en el DOM: agregar/eliminar nodos (`childList`), cambios de atributos (`attributes`) y cambios de texto (`characterData`).

```
const observer = new MutationObserver((mutaciones) => {
  mutaciones.forEach((m) => {
    console.log(`Cambio: ${m.type} en ${m.target.nodeName}`);
  });
});

observer.observe(elemento, {
  childList: true, // detectar nodos agregados/eliminados
  attributes: true, // detectar cambios de clase/atributos
  subtree: true, // observar también los descendientes
});
```

En el ejercicio **a4_443.html** se construyó una To-Do List donde `MutationObserver` registra en tiempo real cada cambio en la lista: cuando se agrega una tarea (`childList`), cuando se elimina, y cuando se marca

5.3 — IntersectionObserver WEB API

■ CONCEPTO

IntersectionObserver detecta cuando un elemento entra o sale del *viewport* (o de otro elemento raíz). Es la API ideal para **lazy loading de imágenes, animaciones on-scroll e infinite scroll**.

Opción	Valor por defecto	Descripción
<code>root</code>	<code>null</code>	Viewport = ventana del navegador
<code>rootMargin</code>	<code>"0px"</code>	Margen para adelantar/retrasar el disparo
<code>threshold</code>	<code>0</code>	% del elemento visible para disparar (0–1)

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      entry.target.classList.add("visible"); // disparar animación CSS
      observer.unobserve(entry.target); // dejar de observar (ya apareció)
    }
  });
}, { root: null, rootMargin: "0px", threshold: 0.2 });

document.querySelectorAll(".caja").forEach((caja) => observer.observe(caja));
```

5.4 — PerformanceObserver WEB API

■ CONCEPTO

PerformanceObserver monitorea métricas de rendimiento de la página en tiempo real: *Long Tasks* (bloqueos del hilo principal >50ms), *Layout Shifts* (inestabilidad visual = CLS) y tiempos de carga de recursos.

Tipo de entrada	Descripción	Métrica relacionada
<code>longtask</code>	Tareas que bloquean el hilo >50ms	TBT (Total Blocking Time)
<code>layout-shift</code>	Desplazamiento visual inesperado	CLS (Cumulative Layout Shift)
<code>navigation</code>	Tiempos de carga de la página	TTFB, DOMContentLoaded
<code>resource</code>	Tiempos de carga de cada recurso	Latencia de red

```
const obs = new PerformanceObserver((list) => {  
  list.getEntries().forEach((entry) => {  
    console.log(entry.entryType, entry.duration);  
  });  
});  
obs.observe({ type: "longtask", buffered: true });
```

Practicá lo aprendido

Ejercicios adicionales para consolidar cada módulo

Módulo 1 — Arrays y métodos funcionales



PROPUESTO 1

Carrito de compras funcional

Dado el siguiente array, realizá las operaciones indicadas encadenando métodos de array:

```
const carrito = [
  { producto: "Remera",    precio: 3500, cantidad: 2, categoria: "ropa"    },
  { producto: "Libro",     precio: 1800, cantidad: 3, categoria: "educacion"},
  { producto: "Zapatilla", precio: 12000, cantidad: 1, categoria: "ropa"    },
  { producto: "Curso JS",  precio: 5000, cantidad: 1, categoria: "educacion"},
  { producto: "Campera",   precio: 8500, cantidad: 1, categoria: "ropa"    },
];
```

1. Calculá el **total del carrito** (precio × cantidad de cada ítem).
2. Obtenés un array con los nombres de los productos de **categoría "ropa"** ordenados por precio descendente.
3. Verificá con `every()` si todos los productos tienen cantidad mayor a 0.
4. Agrupá los productos por categoría usando `reduce()`.
5. Generá un nuevo array con `{ producto, subtotal }` donde subtotal = precio × cantidad.



PROPUESTO 2

Análisis de notas de un curso

```
const curso = [
  { alumno: "Sofía", notas: [9, 8, 10, 7] },
  { alumno: "Marcos", notas: [5, 6, 4, 7] },
  { alumno: "Valentina", notas: [10, 9, 9, 8] },
  { alumno: "Diego", notas: [3, 5, 6, 4] },
];
```

1. Calculá el **promedio** de cada alumno (usá `map()` + `reduce()` anidado).
2. Separalos en **aprobados** (promedio ≥ 6) y **reprobados**.
3. Encontrá al alumno con el **promedio más alto**.
4. Verificá si **alguno** tiene todas las notas ≥ 8 (usá `some()` + `every()`).

Módulo 2 — Matrices



PROPUESTO 3

Transpuesta y rotación de una matriz

1. Escribí una función `transponer(matriz)` que devuelva la **matriz transpuesta** (filas $\rightarrow$ columnas).
Pista: usa `map()` con índice.
2. Escribí una función `rotar90(matriz)` que rote la matriz 90° en sentido horario.
3. Verificá si una matriz cuadrada es **simétrica** (igual a su transpuesta).

Módulo 3 — Objetos



PROPUESTO 4

Sistema de biblioteca

```
const biblioteca = {
  libros: [
    { titulo: "Clean Code", autor: "Martin", anio: 2008, disponible: true },
    { titulo: "JS: The Good", autor: "Crockford", anio: 2008, disponible: false },
    { titulo: "You Don't Know JS", autor: "Simpson", anio: 2015, disponible: true },
  ],
};
```

1. Usá `Object.entries()` para mostrar todas las propiedades del primer libro.
2. Filtrá los **libros disponibles** y mostrá sus títulos.
3. Agrupá los libros por año con `reduce()`.
4. Implementá un método `prestar(titulo)` que marque un libro como no disponible.

Módulo 4 — Funciones de orden superior



PROPUESTO 5

Pipeline de transformaciones

1. Creá una función `pipeline(...fns)` que reciba cualquier cantidad de funciones y las aplique en cadena sobre un valor inicial.
2. Usala para convertir `" hola mundo "` → trim → capitalize → encriptar con Caesar.
3. Implementá una función `memoizar(fn)` que cachee los resultados de una función para evitar recalcularlos.

Módulo 5 — Observers



PROPUESTO 6

Observer con historial

1. Extendé la clase `Subject` del ejercicio 5.1 para que guarde un **historial de todos los eventos** notificados.
2. Agregá un método `replay(observer)` que envíe todos los eventos históricos a un nuevo suscriptor.
3. Agregá soporte para **prioridad**: algunos observers se notifican antes que otros según un número de prioridad.



PROPUESTO 7

Lazy loading de imágenes

1. Creá una página HTML con 10 imágenes que tengan sus URLs en el atributo `data-src` en lugar de `src`.
2. Usá `IntersectionObserver` para cargar cada imagen solo cuando esté a punto de ser vista (threshold: 0.1).
3. Mostrá un placeholder gris mientras la imagen no está cargada.
4. Usá `observer.unobserve()` una vez que la imagen cargó.

¡Seguí practicando!

El dominio del código viene con la práctica constante.
Cada ejercicio que completás te acerca a ser un
desarrollador más sólido.

Programación Web · JavaScript Avanzado